

ENTERPRISE DASHBOARD

Real-time Analytics & Advanced State Management

Week 10 Project Documentation

Project	Enterprise Dashboard with Real-time Analytics
Frontend	React 18 + TypeScript + Vite
State	Redux Toolkit + Redux Persist + React Query
UI / Charts	Material UI + Recharts
Routing	React Router v6 with protected & nested routes
Real-time	WebSocket service with reconnect handling
Testing	Vitest + React Testing Library

This documentation is based on the supplied Week 10 project requirements and the generated project implementation.

1. Project Overview

The project is an Enterprise Dashboard with Real-time Analytics and Advanced State Management. The Week 10 specification calls for a scalable React architecture with TypeScript, Redux Toolkit, React Router, data visualization, WebSocket updates, testing, Storybook, performance optimization, internationalization and PWA capabilities. The supplied project implements the core dashboard workflow, protected routing, feature-based organization, state management, charts, a local real-time stream, testing and CI-ready structure.

2. Key Features

- Code splitting using React.lazy and Suspense for route-level loading.
- Redux Toolkit slices for authentication, dashboard data and notifications.
- Redux Persist for selected client state persistence.
- Protected dashboard routes and nested navigation with React Router v6.
- Interactive analytics using Recharts.
- Material UI component system with responsive layouts.
- React Query configured for server-state caching.
- WebSocket service abstraction with periodic real-time updates and reconnect handling.
- CSV report export from the Reports page.
- Vitest + React Testing Library test example and Storybook component example.
- GitHub Actions workflow for type-check, test and production build.

3. Technical Architecture

Layer	Implementation
Presentation	React 18, TypeScript, Material UI, reusable atomic components
Routing	React Router v6, protected dashboard route, nested child routes
State	Redux Toolkit, Redux Persist, typed hooks
Server state	TanStack React Query configuration
Analytics	Recharts line and bar charts
Real-time	WebSocketService abstraction with reconnect method
Quality	Vitest, React Testing Library, Storybook, ESLint-ready structure
CI/CD	GitHub Actions workflow for install, type-check, test and build

Folder structure

src/app — store, providers and typed hooks
src/features — auth, dashboard and notifications
src/components — atoms, organisms and templates
src/pages — dashboard, analytics, reports and settings
src/services — WebSocket integration
src/hooks — reusable application hooks
.github/workflows — CI automation

4. Application Screenshots

The following screenshots represent the implemented dashboard screens and are included as visual evidence for the submission.

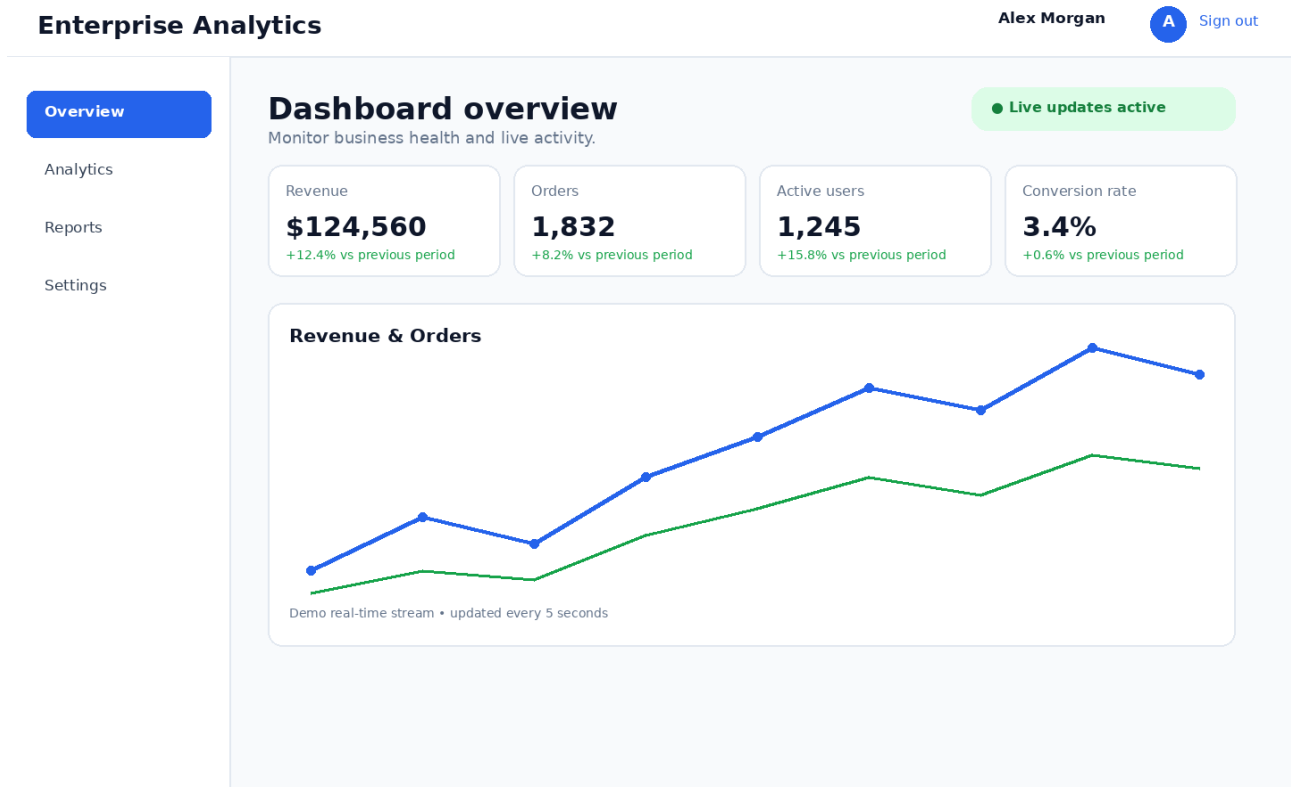


Figure 1 — Dashboard overview with KPI cards and live revenue/orders chart.

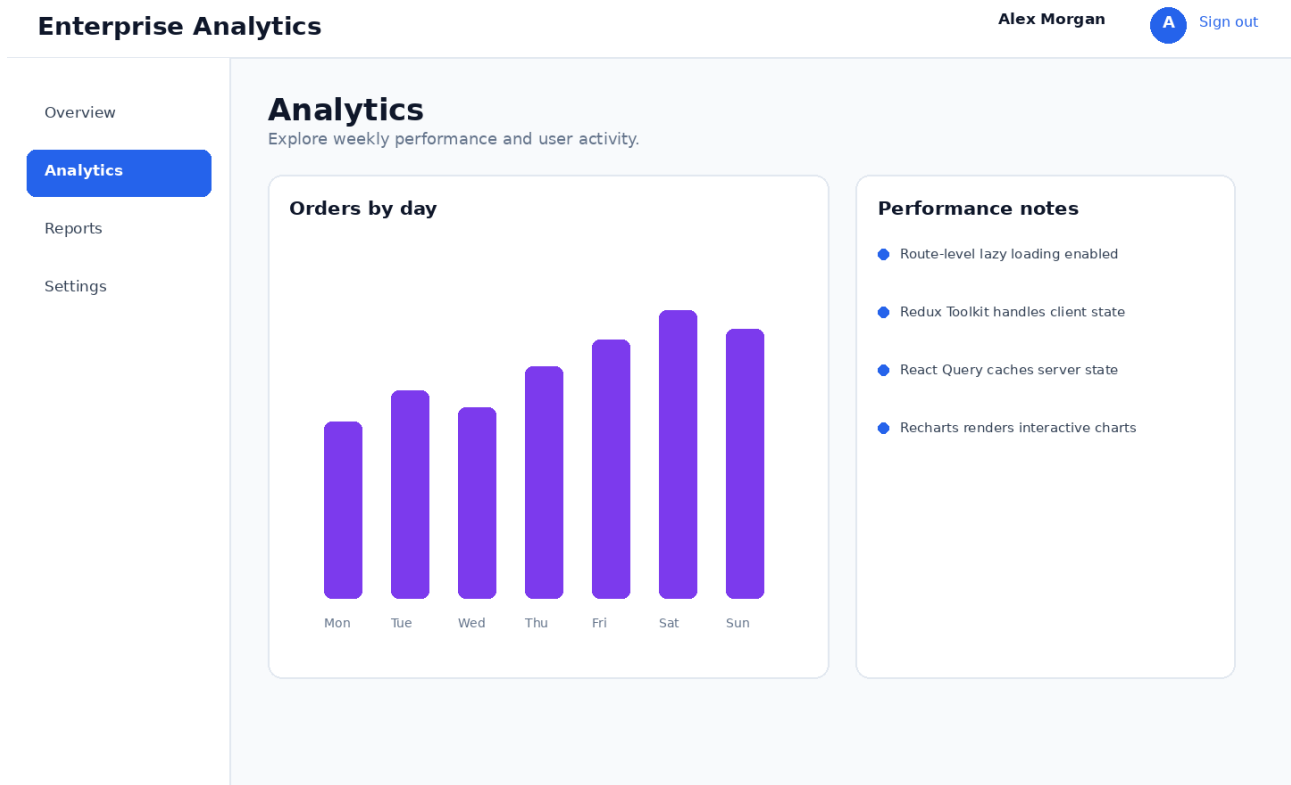


Figure 2 — Analytics page with orders visualization and architecture notes.

5. Application Screenshots (continued)

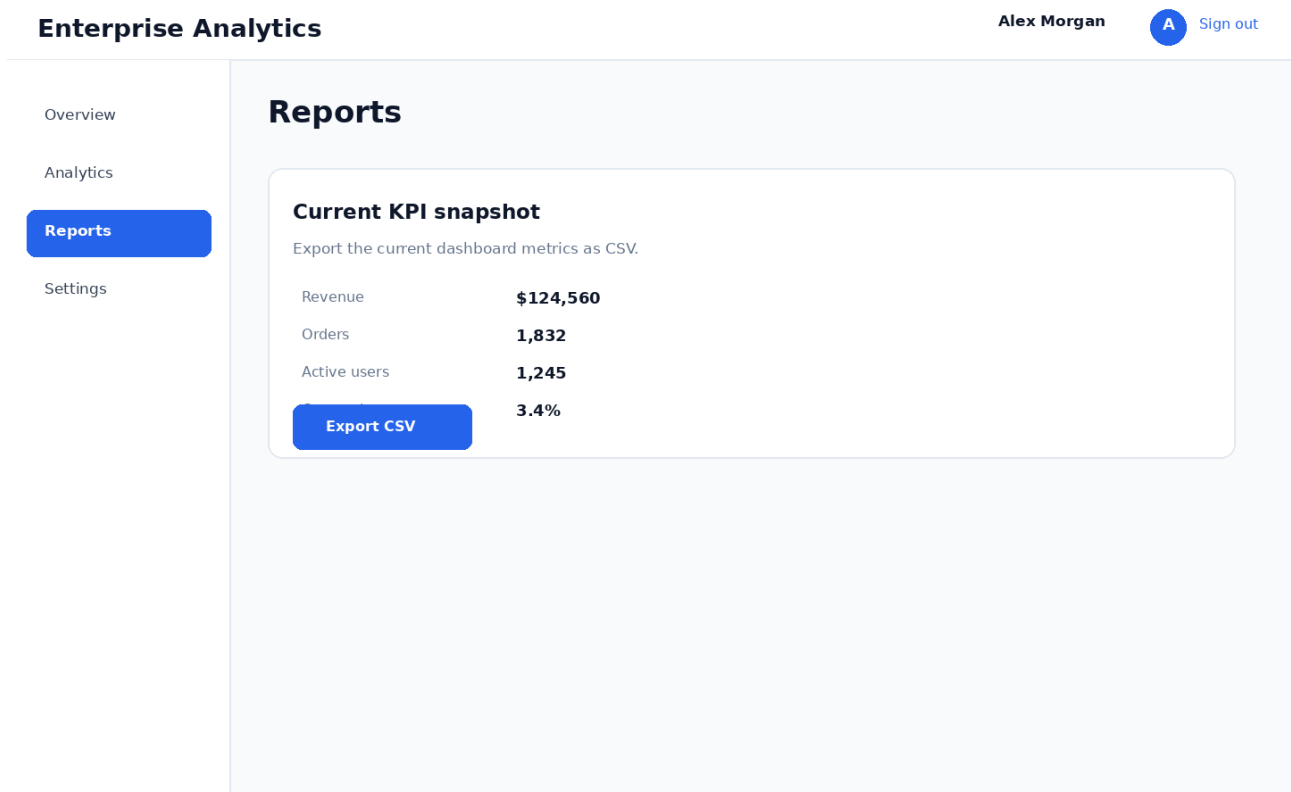


Figure 3 — Reports page with KPI snapshot and CSV export.

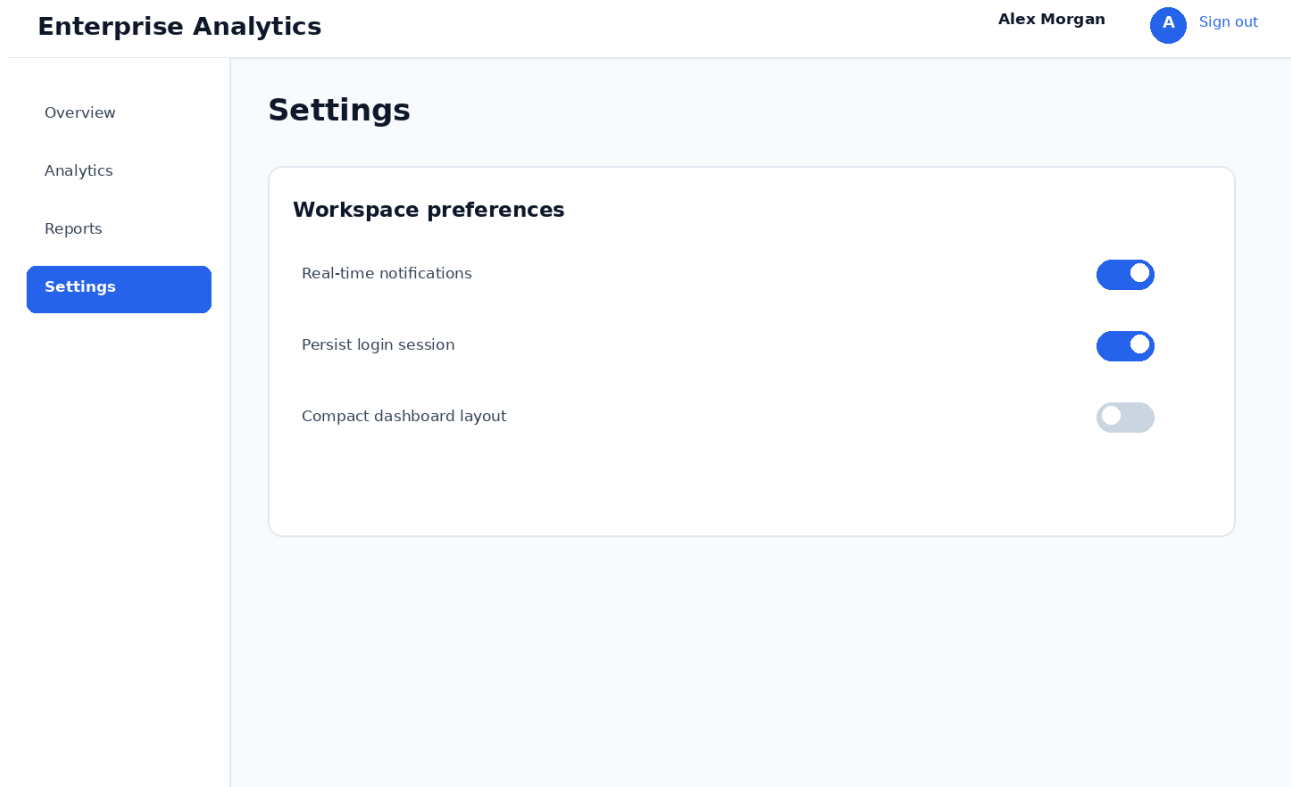


Figure 4 — Workspace settings and notification preferences.

6. Authentication & Architecture Screenshots

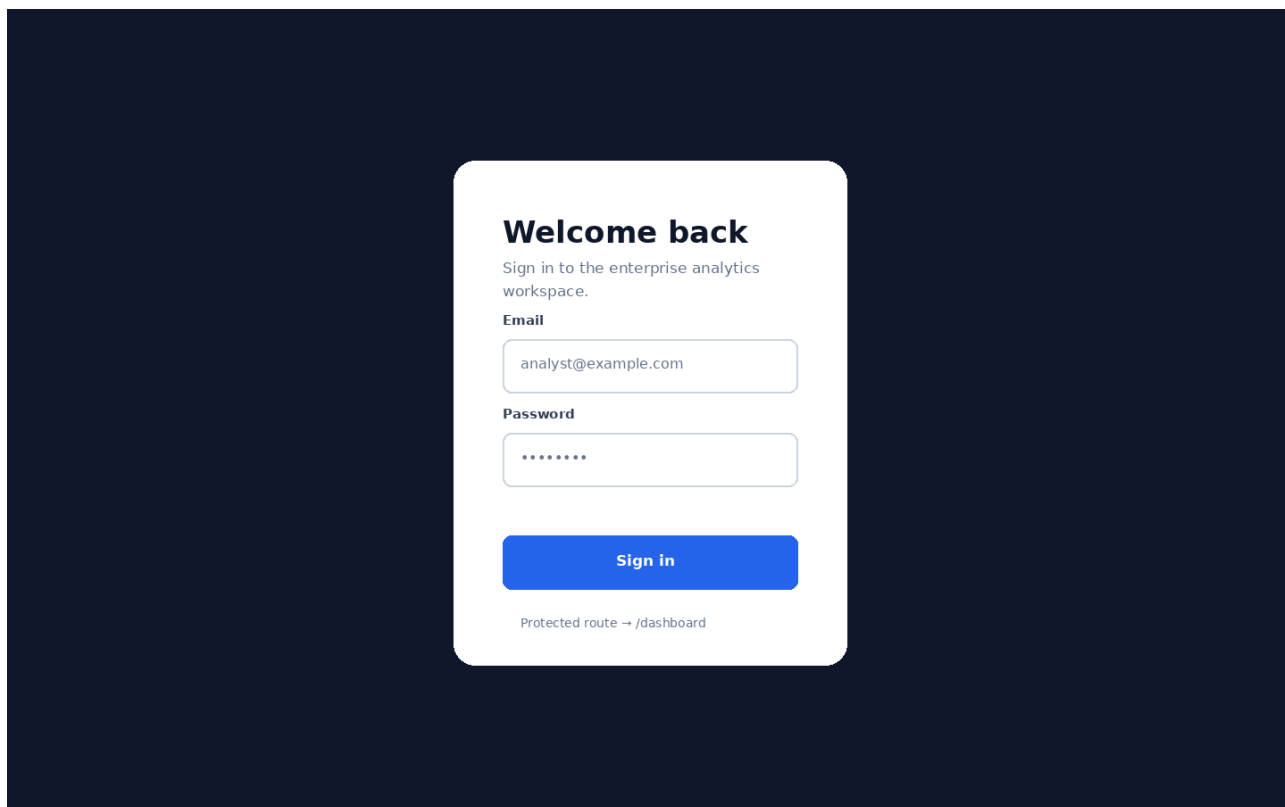


Figure 5 — Login screen and protected-route entry flow.

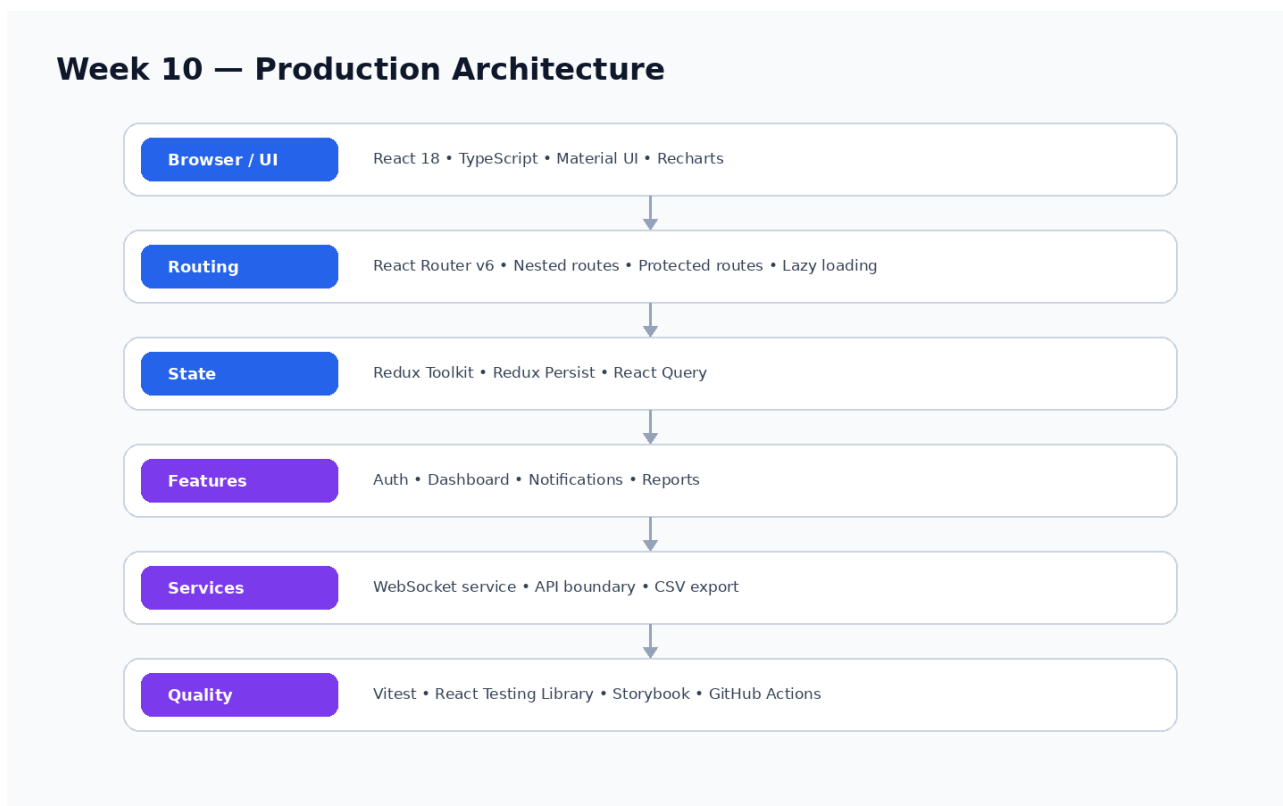


Figure 6 — Project architecture and data-flow layers.

7. Installation & Usage

Prerequisites: Node.js and npm. From the project root run **npm install**, then **npm run dev**. Use **npm run type-check** for TypeScript validation, **npm test** for the test suite and **npm run build** for a production build. The local dashboard includes a demo authentication flow; production authentication should be connected to a secure backend.

Quality and deployment checklist

- Keep secrets out of source control and use environment variables.
- Use authenticated WSS endpoints for production real-time communication.
- Validate server-sent messages at the application boundary.
- Run type-check, tests and production build in CI before deployment.
- Use route-level code splitting and React profiling to control bundle and render cost.
- For full production readiness, add the remaining specified i18n and PWA layers as required by the assignment.

8. Conclusion

The Week 10 project demonstrates the requested modern frontend architecture through a working enterprise dashboard structure. It combines TypeScript, Redux Toolkit, protected nested routing, interactive analytics, real-time update handling, reusable components, testing and CI/CD foundations. The included screenshots document the major user-facing workflows for submission.